

LPI-701-100 · DevOps Tools Engineer

Kompakt-Referenz · Merksätze, Befehle und Verwechslungspaare aus dem 3-Tage-Plan · Bestehensgrenze ca. 500/800 Punkte (≈65 %)

1 · Die Vokabel-Matrix

Der häufigste Verwechslungstest der Prüfung. Zuerst lernen.

Tool	Dateien	Sprache	Facts	Modell
Puppet	Manifests <code>.pp</code> , Module	eigene DSL =>	Factor	Pull (Agent holt Katalog, ~30 min)
Chef	Recipes in Cookbooks	Ruby <code>do...end</code>	Ohai	Pull (Agent)
Ansible	Playbooks, Rollen	YAML	Setup-Modul	Push via SSH, agentless

Format	Sprache
Vagrantfile	Ruby
Packer-Template	HCL (früher JSON)
Terraform	HCL
Compose / K8s / Playbook	YAML
Jenkinsfile	Groovy
Helm	Charts + Values (YAML)

ESELSBRÜCKE
Chef → Rezepte im Kochbuch, und der Koch sagt „Ohai“. Puppet → Marionette, die vom Master gezogen wird (Pull), Fakten von Factor.

2 · Git

cherry-pick	einzelnen Commit übernehmen
merge	Historien zusammenführen (Merge-Commit)
rebase	Commits auf neue Basis, lineare Historie
pull	= <code>fetch</code> + <code>merge</code> (mit <code>--rebase</code> : rebase)
stash	Änderungen temporär beiseitelegen, <code>stash pop</code>
tag -a	annotierter Tag; Push nur mit <code>--tags</code>
reset	<code>--soft</code> Index behalten · <code>--hard</code> alles verwerfen
revert	Gegen-Commit (sicher für geteilte Branches)

GitFlow: `main` = Produktion · `develop` = Integration · Feature-Branches von develop · Hotfix von main.

FALLEN
Rebase nie auf bereits gepushten, geteilten Branches. — `.gitignore` wirkt nur auf *unversionierte* Dateien; bereits getrackte: `git rm --cached <datei>`. — `git checkout <hash>` → **DETACHED HEAD**
.

3 · CI/CD-Konzepte

Cont. Integration	häufiges Mergen + automatische Builds/Tests
Cont. Delivery	jederzeit releasefähig , Prod-Deploy manuell freigegeben
Cont. Deployment	jeder grüne Build geht automatisch in Prod

Strategie	Kern	Downtime
Rolling	Instanzen nacheinander ersetzen (K8s-Default)	nein
Blue-Green	2 volle Umgebungen, Traffic umschalten; sofortiges Rollback, doppelte Kosten	nein
Canary	schrittweise Traffic-Erhöhung (5 % → ...)	nein
Recreate	alles stoppen, dann neu starten	JA

Webhook = Git-Server ruft CI aktiv (Push) | **Polling** = CI fragt periodisch nach.

4 · Jenkins Declarative Pipeline

```
pipeline {
    agent any                // none | {label 'x'} | {docker{image 'maven'}}
    tools {
        maven 'M3'; jdk 'JDK11'
    }
    environment {
        REG='reg.firma.de'
        TOK=credentials('sonar-token')
    }
    options {
        timeout(time:30, unit:'MINUTES')
        buildDiscarder(logRotator(numToKeepStr:'10'))
    }
    parameters {
        string(name:'BRANCH', defaultValue:'main')
        booleanParam(name:'DEPLOY', defaultValue:false)
        choice(name:'ENV', choices:['dev','prod'])
    }
    triggers {
        pollSCM('H/5 * * * *') // nur bei Änderung
        cron('H 2 * * *') // stur nach Zeit
    }
    stages {
        stage('Build') { steps { sh 'mvn -B clean package' } }
        stage('Test') { parallel {
            stage('Unit'){ steps{ sh 'mvn test' } }
            stage('IT') { steps{ sh 'mvn verify' } } } }
        stage('Deploy') {
            when { branch 'main' }
            steps { input message:'Deploy nach Prod?'
                    sh './deploy.sh' }
        }
    }
    post {
        always { junit 'target/surefire-reports/*.xml'
                  archiveArtifacts artifacts:'target/*.jar' }
        success { echo 'ok' }
        failure { mail to:'team@firma.de', subject:'FAIL' }
        unstable{ echo 'flaky' }
        cleanup { deleteDir() }
    }
}
```

Declarative	startet mit <code>pipeline { }</code> , strukturiert, validiert
Scripted	startet mit <code>node { }</code> , freier Groovy-Code
Controller	orchestriert; Agent führt Builds aus
script { }	Groovy-Ausweg innerhalb von Declarative

PRÜFUNGSFALLEN

`archiveArtifacts` =

DAUERHAFT

am Build (Download im UI). `stash/unstash` = nur

TEMPORÄR

zwischen Stages/Agents desselben Builds.

`when` = bedingte Stage (auch `expression`, `allOf`, `not`). `only`: wäre GitLab-CI!

`input` = manuelle Freigabe (Continuous *Delivery*). Kein `pause()`.

Top-Level-Direktiven: `agent`, `environment`, `options`, `parameters`, `triggers`, `tools`, `stages`, `post`. Kein `containers`.

5 · Docker

Dockerfile

FROM	Basisimage; mehrfach = Multi-Stage
RUN	Befehl beim Build , erzeugt Layer
CMD	Default-Argumente - von <code>docker run</code> überschreibbar
ENTRYPOINT	festes Kommando - nicht überschrieben
COPY	Dateien kopieren (bevorzugen)
ADD	wie COPY + URLs + tar-Auto-Entpacken
EXPOSE	nur Dokumentation - veröffentlicht nichts!
USER	unprivilegierter Benutzer (Security)
ENV / ARG	Laufzeit-Variable / nur Build-Zeit

Befehle

```
docker run -p HOST:CONTAINER -d --name web nginx
docker ps -a                # auch gestoppte
docker logs [-f] web        # auch bei Exited!
docker exec -it web /bin/bash # in LAUFENDEN Container
docker inspect web          # volles JSON (Container+Image)
docker inspect --format '{{.NetworkSettings.IPAddress}}' web
docker build -t myapp:1.0 .
docker tag myapp:1.0 reg.firma.de/myapp:1.0
docker push reg.firma.de/myapp:1.0
docker volume create daten ; docker rm/rmi/prune
```

FALLEN

`-p 8080:80` =

HOST:CONTAINER

. — Registry-Adresse steckt

IM IMAGE-NAMEN

: erst `tag`, dann `push`. — `-v /pfad:/ziel` (Slash!) = Bind-Mount, `-v name:/ziel` = benanntes Volume. — Ohne Tag zieht Docker `:latest` (≠ neueste Version). —

`exec` = laufender Container, `run` = neuer Container.

LAYER-CACHE

Selten geänderte Schritte (Dependencies) nach oben, häufig geänderte (Quellcode-COPY) nach unten. `.dockerignore` hält Build-Kontext klein.

6 · Compose & Swarm

```
services:
  web:
    build: ./app          # baut lokal
    image: myapp:1.0      # Tag fürs Ergebnis
    ports: ["8080:80"]
    env_file: .env.prod
    environment: [LOG_LEVEL=debug]
    depends_on:
      db: { condition: service_healthy }
  db:
    image: postgres:16
    volumes: [pgdata:/var/lib/postgresql/data]
    healthcheck: { test: ["CMD", "pg_isready"] }
volumes: { pgdata: }
```

```
docker compose up -d | down [-v] | logs -f | ps | build
docker swarm init ; docker swarm join-token worker
docker service create --name web --replicas 3 -p 8080:80 nginx
docker service scale web=5 | update --replicas 5 web
docker service update --publish-add 8080:80 web
docker service rollback web | docker service ps web
docker stack deploy -c compose.yml app ; docker node ls
```

Routing Mesh	publizierter Port ist auf jedem Node erreichbar, LB intern
overlay	Netz über mehrere Nodes (bridge = nur ein Host)
global	1 Task pro Node (= K8s DaemonSet); Default: replicated
Quorum	3 Manager → 1 Ausfall tolerierbar; (N−1)/2 (Raft)

FALLEN

`depends_on` (Kurzform) = nur

STARTREIHENFOLGE

, keine Bereitschaft → `condition: service_healthy` + healthcheck. — `down -v` löscht

VOLUMES

(Datenverlust). — `stack deploy`

IGNORIERT

`build:` → Images müssen in der Registry liegen. — Servicenamen sind per DNS auflösbar (`jdbc:.../db:5432`).

7 · Kubernetes

Pod	kleinste deploybare Einheit (1..n Container, geteiltes Netz)
ReplicaSet	hält Soll-Anzahl Pods
Deployment	ReplicaSet + Rolling Update + Rollback
DaemonSet	1 Pod pro Node (Log/Monitoring-Agents)
StatefulSet	stabile Identität + eigener Storage (DBs)
Job / CronJob	einmalig / zeitgesteuert
ConfigMap	unkritische Konfiguration
Secret	sensibel, nur base64-kodiert , nicht verschlüsselt
PV	Speicher-Ressource (Angebot, clusterweit)
PVC	Anforderung (Nachfrage, namespace) – Pod mountet den PVC
Namespace	logische Trennung + Quotas + RBAC (OpenShift: Projekt)

Service-Typ	Bedeutung
ClusterIP	nur clusterintern (Default)
NodePort	fester Port auf jedem Node
LoadBalancer	externer Cloud-LB (baut auf NodePort auf)
Ingress / Route	HTTP-Routing per Hostname/Pfad

```
kubectl apply -f app.yml      # deklarativ, wiederholbar
kubectl create -f app.yml    # scheitert, wenn vorhanden
kubectl get pods -n cte-sandbox [-o wide]
kubectl describe pod <n>     # ← EVENTS am Ende!
kubectl logs -f <pod> [-c container] [--previous]
kubectl set image deployment/web app=web:2.0
kubectl rollout status|undo deployment/web
kubectl scale deployment/web --replicas=5
kubectl exec -it <pod> -- /bin/bash
```

Liveness	fehlgeschlagen → Container- Restart
Readiness	fehlgeschlagen → kein Traffic (aus Endpoints)
Startup	schützt langsam startende Apps

DIAGNOSE-REGEL

PENDING

= noch kein Container → `describe` / `get events` (logs und top liefern nichts!).

CRASHLOOPBACKOFF

= startet und stirbt → `logs --previous`.

IMAGEPULLBACKOFF

= Image/Registry/Secret prüfen.

DNS: `<service>.<namespace>.svc.cluster.local` (im selben NS reicht der Kurzname).

8 · Vagrant · Packer · Cloud-Init

Vagrant	startet VMs aus fertigen Boxen (Entwicklungsumgebung)
Packer	baut Images/Boxen (Golden Image, Immutable Infra)
Cloud-Init	konfiguriert VM beim ersten Boot (User-Data)

Vagrantfile = RUBY

```

Vagrant.configure("2") do |config|
  config.vm.box = "ubuntu/jammy64"
  config.vm.network "forwarded_port", guest:80, host:8080
  config.vm.synced_folder ".", "/vagrant"
  config.vm.provision "ansible" do |a| a.playbook="site.yml" end
end

```

up (erstellen+starten+provisionieren) · ssh · halt · destroy · provision (nur Provisioner) · reload · box add/list · status

Provider = wo (VirtualBox, libvirt, Hyper-V) | Provisioner = wie (Shell, Ansible, Puppet, Chef). Default-Synced-Folder: /vagrant. Box = Template.

Packer

Bausteine: Builders (Zielformat) · Provisioners (Konfiguration) · Post-Processors (Nachbearbeitung). Befehle: packer build|validate|fmt.

Cloud-Init

```

#cloud-config
hostname: web01
users: [{name: horst, ssh_authorized_keys: ["ssh-rsa AAA..."]}]]
packages: [nginx]
write_files: [{path: /etc/motd, content: "Hallo"}]
runcmd: ["systemctl enable --now nginx"]

```

Läuft nur beim ersten Boot. Alternativ Shell-Skript als User-Data (beginnt mit #!).

9 · Ansible

```

- hosts: webserver
  become: true
  vars: { port: 8080 }
  tasks:
    - name: nginx installieren
      package: { name: nginx, state: present }
    - name: Konfiguration ausrollen
      template: { src: nginx.conf.j2, dest: /etc/nginx/nginx.conf }
      notify: restart nginx
  handlers:
    - name: restart nginx
      service: { name: nginx, state: restarted }

```

```

ansible all -m ping
ansible web -m shell -a "uptime" -b
ansible-playbook site.yml -i inv --check --limit web
ansible-vault encrypt|edit|view secrets.yml
ansible-galaxy install geerlinguy.nginx

```

Inventory	Hosts+Gruppen (INI/YAML), Default /etc/ansible/hosts
Handler	läuft per notify nur bei Änderung, am Play-Ende
Rolle	tasks/ handlers/ templates/ defaults/ vars/ files/
Vault	Verschlüsselung sensibler Daten
--check	Dry-Run

Idempotenz: Module beschreiben den Soll-Zustand; Änderung nur bei Abweichung (ok vs. changed).

10 · Puppet

```

package { 'nginx':
  ensure => installed,
}
service { 'nginx':
  ensure => running,
  enable => true,
  require => Package['nginx'],
}
file { ['/etc/motd']:
  ensure => file,
  content => "Hallo\n",
}

```

Manifests (.pp) in Modulen · Classes · Factor liefert Facts · Hiera = Daten-Backend · Agent holt kompilierten Katalog vom Puppet Server (Pull, ~30 min) · puppet apply manifest.pp lokal.

11 · Monitoring & Logging

Prometheus	Pull: scrapt /metrics per HTTP; Sprache PromQL
Pushgateway	Ausnahme für kurzlebige Jobs
Alertmanager	Alerts dedupliziert/gruppiert/routet (Mail, Chat)
Grafana	Visualisierung/Dashboards

Exporter	Liefert
Node Exporter	Host: CPU, RAM, Disk, Netz
cAdvisor	Container-Metriken
Blackbox	Prüfung von außen (HTTP, ICMP, TCP)
SNMP	Netzwerkgeräte

ELK	Elasticsearch (Speicher/Suche) · Logstash (Pipeline) · Kibana (Visualisierung)
Logstash	Pipeline: Input → Filter (grok) → Output
Beats	schlanke Shipper: Filebeat (Logs), Metricbeat, Packetbeat
Fluentd	Alternative zu Logstash (CNCF, in K8s verbreitet)

12 · Moderne Softwareentwicklung

Code	Bedeutung
200 / 201	OK / Created
301 / 302	Redirect (permanent/temporär)
400	Bad Request
401	nicht authentifiziert (Login fehlt/falsch)
403	authentifiziert, aber keine Rechte
404 / 500	Not Found / Server Error

Verben: GET (lesen) · POST (erzeugen, **nicht idempotent**) · PUT (ersetzen) · PATCH (teilweise) · DELETE. Idempotent: GET, PUT, DELETE — **nicht** POST.
REST = stateless: jeder Request enthält alles, keine Server-Session. `Content-Type: application/json`. **CORS** = Browser-Schutz, Server erlaubt fremde Origins per Header. **API-Versionierung** (`/api/v1/`) sichert Abwärtskompatibilität.

12-Factor (Prüfungskern): Config in **Umgebungsvariablen** · Backing Services austauschbar · Build/Release/Run trennen · Prozesse **stateless** (Session in Redis/DB!) · Logs als Event-Stream nach **stdout**.

Microservices: unabhängig deploybar/skalierbar, eigene Datenhaltung, API- oder Message-Broker-Kommunikation (RabbitMQ, Kafka = entkoppelt/asynchron). Preis: verteilte Komplexität, Latenz, schwierigeres Debugging. Gemeinsame DB-Tabellen = Anti-Pattern.

Immutable Infrastructure: nie in-place patchen – neu bauen und austauschen.

13 · Container-Security & Discovery

Namespaces	Isolation der Sicht: PID, NET, MNT, UTS, IPC, USER
Cgroups	Ressourcen-Limits: CPU, RAM, I/O (→ OOMKilled)
Capabilities	aufgeteilte Root-Rechte: <code>--cap-drop ALL --cap-add NET_BIND_SERVICE</code>
seccomp	Syscall-Filter; dazu SELinux/AppArmor

Best Practices: nicht als root (**USER**), `--read-only`, keine Secrets im Image, schlanke Basisimages (alpine/distroless), Images scannen mit **Trivy** oder **Clair**, vertrauenswürdige Registries.

etcd	verteilter KV-Store, Herz von Kubernetes (Raft)
Consul	Discovery + Health Checks + DNS-Interface
ZooKeeper	Klassiker (Hadoop/Kafka-Umfeld)

14 · Prüfungstaktik

- **Fragetypen:** Single Choice · Multiple Response (Anzahl steht dabei, **alles-oder-nichts**) · Fill-in-the-blank (exakte Schreibweise, meist ohne `docker/kubectl`-Präfix, Optionen selten nötig).
- **Zeit:** 60 Fragen / 90 Min = 90 s pro Frage. Ihr Tempo lag bei ~22 s → reichlich Puffer für einen zweiten Durchgang.
- Unsichere Fragen markieren, weitergehen, am Ende zurückkommen. **Nie unbeantwortet lassen** – kein Punktabzug für falsche Antworten.
- Bei Multiple Response zuerst die offensichtlich falschen Optionen streichen; die Anzahl ist ein Hinweis auf die Struktur.
- Erfundene Befehle sind ein häufiges Ablenkungsmuster (`docker service undo`, `kubectl show`, `vagrant update`) – wirkt der Befehl unbekannt, ist er meist erfunden.
- Achten Sie auf Tool-Verwechslungen zwischen den Ökosystemen (GitLab-CI-Syntax in einer Jenkins-Frage, Chef-Vokabular in einer Puppet-Frage).

LETZTE STUNDE VOR DER PRÜFUNG

Nur noch Abschnitt 1 (Vokabel-Matrix) und die rot markierten Fallen-Kästen lesen. Nichts Neues mehr anfangen.

Erstellt als Abschluss des 3-Tage-Plans · Ergebnisse: Diagnostik 85 % → Drill 94 % → Neue Themen 90 % → Vollsimulation 85 % → Exam-Format 88 %